



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

MACHINE LEARNING
ROSAS CARRILLO BARY SHARED

Practica 3. Perceptron

Corro Mendoza Onasis Alejandro 2022630202
Herrera Mauricio Pedro Alonso 2020600448

Ciudad de México
09 de June de 2026

Índice

1. Introducción	1
2. Marco teórico	2
2.1. El perceptrón como clasificador lineal	2
2.2. Aprendizaje supervisado y función de pérdida	2
2.3. Regla delta del perceptrón clásico	2
2.4. Descenso por gradiente (Adaline)	3
2.5. Particle Swarm Optimization (PSO)	3
2.6. Normalización de las entradas	4
3. Desarrollo: implementaciones y pruebas	5
3.1. Datos y protocolo experimental	5
3.2. Implementación 1: Perceptrón clásico	5
3.3. Implementación 2: Perceptrón con Descenso por Gradiente	5
3.4. Implementación 3: Perceptrón con PSO	6
3.5. Resultados	6
3.6. Análisis	7
4. Conclusiones	8

1. Introducción

El perceptrón es el modelo más sencillo que se puede llamar, sin exagerar, una «neurona artificial». Lo propuso Frank Rosenblatt en 1958 y casi todas las redes neuronales modernas empiezan reciclando su idea: sumar entradas con pesos, aplicar una función de activación y comparar contra una etiqueta. Para una carrera de Sistemas Computacionales sirve para ver, con poco código, cómo un programa pasa de tener reglas escritas a mano a derivarlas de los datos.

Este reporte compara tres maneras de entrenar el mismo perceptrón sobre el mismo problema (clasificación binaria de tumores: maligno vs. benigno, dataset *Breast Cancer Wisconsin* de UCI / Kaggle):

1. Perceptrón clásico con la regla delta de Rosenblatt.
2. Perceptrón con descenso por gradiente, en la variante Adaline.
3. Perceptrón con Particle Swarm Optimization (PSO), sin gradientes.

La idea es que quien lo lea entienda primero las matemáticas y después qué cambia entre cada estrategia: velocidad, estabilidad, exactitud y costo.

2. Marco teórico

2.1. El perceptrón como clasificador lineal

Dado un vector de entrada $\mathbf{x} = (x_1, x_2, \dots, x_n)^T \in \mathbb{R}^n$, el perceptrón calcula una salida lineal (también llamada *net input*) como la suma ponderada de las entradas más un sesgo b :

$$z(\mathbf{x}) = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

Después aplica una función de activación $\varphi(\cdot)$ a z . En el perceptrón clásico esa función es el escalón de Heaviside:

$$\varphi(z) = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{si } z \leq 0 \end{cases}$$

Geométricamente, $\mathbf{w}^T \mathbf{x} + b = 0$ es un hiperplano en $\mathbb{R}^n$ que parte el espacio en dos. Lo que cae de un lado es clase 1, lo del otro es clase 0. Por eso el perceptrón sólo resuelve problemas *linealmente separables*: necesita que exista al menos un hiperplano capaz de dejar todos los positivos a un lado y todos los negativos al otro. Si los datos están entreverados (el clásico XOR, por ejemplo), ningún ajuste de pesos lo va a salvar.

$$\mathbf{w}^T \mathbf{x} + b = 0$$

hiperplano de decisión

Figura 1: El perceptrón aprende un hiperplano que separa las dos clases en el espacio de características.

2.2. Aprendizaje supervisado y función de pérdida

Sea un conjunto de entrenamiento $D = \{(\mathbf{x}^{(k)}, y^{(k)})\}_{k=1}^N$ con etiquetas $y^{(k)} \in \{0, 1\}$. Aprender es encontrar el vector $\mathbf{w}^* \in \mathbb{R}^{n+1}$ (incluyendo el bias) que minimiza alguna función de pérdida $J(\mathbf{w})$. Ahí es donde se separan las tres implementaciones de este reporte: cada una minimiza algo distinto, con un algoritmo distinto.

Implementación	Función de pérdida	Optimizador
Clásico	Errores de clasificación (regla delta)	Actualización online
GD / Adaline	Suma de errores al cuadrado (SSE)	Descenso por gradiente batch
PSO	Número de muestras mal clasificadas	Enjambre de partículas

2.3. Regla delta del perceptrón clásico

Para cada muestra $(\mathbf{x}, y)$ se calcula la predicción $\hat{y} = \varphi(z)$ y se actualizan los pesos sólo si hubo error:

$$\Delta w_i = \eta(y - \hat{y})x_i$$

$$w_i \leftarrow w_i + \Delta w_i$$

con $\eta \in (0, 1]$ la tasa de aprendizaje. La regla es bastante intuitiva: si la red predijo 0 y debía ser 1, los pesos se mueven en la dirección de $\mathbf{x}$; si predijo 1 y debía ser 0, se mueven en sentido opuesto. El bias se actualiza con $\Delta b = \eta(y - \hat{y})$.

Rosenblatt demostró que, si los datos son linealmente separables, esta regla converge en un número finito de pasos (Teorema de Convergencia del Perceptrón). Si no lo son, oscila para siempre.

2.4. Descenso por gradiente (Adaline)

La variante Adaline (Widrow & Hoff, 1960) cambia algo aparentemente pequeño pero con consecuencias grandes: durante el entrenamiento usa la salida lineal $\varphi(z) = z$ en lugar del escalón, y define una pérdida diferenciable, la suma de errores al cuadrado:

$$J(\mathbf{w}) = \frac{1}{2} \sum_{k=1}^N (y^{(k)} - \varphi(z^{(k)}))^2$$

Como J es diferenciable, se puede calcular su gradiente respecto a cada peso:

$$\frac{\partial J}{\partial w_i} = - \sum_{k=1}^N (y^{(k)} - \varphi(z^{(k)})) x_i^{(k)}$$

y aplicar descenso por gradiente en sentido opuesto:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w}) = \mathbf{w} + \eta \sum_k (y^{(k)} - z^{(k)}) \mathbf{x}^{(k)}$$

En la implementación se entrena con etiquetas en $\{-1, +1\}$ y no en $\{0, 1\}$. La razón: el umbral de decisión es $z > 0$. Con etiquetas simétricas el gradiente empuja la salida lineal hacia el lado correcto del cero; con $\{0, 1\}$ tira hacia 0.5, que no es la frontera. La suma también se promedia por N para que η no dependa del tamaño del lote.

2.5. Particle Swarm Optimization (PSO)

PSO es un algoritmo metaheurístico inspirado en bandadas de aves. En vez de calcular un gradiente se mantiene un enjambre de P partículas: cada partícula i es un candidato a vector de pesos $\mathbf{w}^{(i)}$ moviéndose por el espacio de búsqueda con velocidad $\mathbf{v}^{(i)}$.

Cada partícula recuerda dos cosas:

- $\mathbf{p}^{(i)}$: su mejor posición personal hasta ahora (*pbest*).
- $\mathbf{g}$: la mejor posición que encontró el enjambre completo (*gbest*).

En cada iteración, la velocidad se actualiza así:

$$\mathbf{v}^{(i)} \leftarrow \omega \mathbf{v}^{(i)} + c_1 r_1 (\mathbf{p}^{(i)} - \mathbf{w}^{(i)}) + c_2 r_2 (\mathbf{g} - \mathbf{w}^{(i)})$$

y la posición:

$$\mathbf{w}^{(i)} \leftarrow \mathbf{w}^{(i)} + \mathbf{v}^{(i)}$$

donde ω es la inercia (cuánta velocidad anterior conserva la partícula), c_1 el coeficiente cognitivo (atracción a su mejor personal), c_2 el coeficiente social (atracción al mejor global) y $r_1, r_2 \sim \text{Uniforme}(0, 1)$ son ruido estocástico. La función a minimizar (*fitness*) es directamente el número de muestras mal clasificadas. No necesita ser diferenciable, lo cual es la gracia.

2.6. Normalización de las entradas

Las tres implementaciones aplican `StandardScaler`, que lleva cada característica x_i a media cero y desviación estándar uno:

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}$$

Es necesario porque las características viven en escalas muy diferentes. En el dataset, por ejemplo, el área de la célula está en cientos y la textura en unidades. Sin normalizar, las features grandes dominan los pesos sólo por su escala, no porque discriminen mejor.

3. Desarrollo: implementaciones y pruebas

3.1. Datos y protocolo experimental

Dataset: *Breast Cancer Wisconsin (Diagnostic)*, descargado vía kagglehub desde uciml/breast-cancer-wisconsin-data. Son 569 muestras de núcleos celulares con 30 características numéricas (radio medio, textura, perímetro, área, suavidad, etc.) y una etiqueta binaria: M (maligno, codificado como 1) o B (benigno, codificado como 0). Se descarta la columna id y la columna fantasma Unnamed: 32 (toda NaN).

Protocolo: las tres implementaciones reciben los mismos $\mathbf{X}$, $\mathbf{y}$, comparten el StandardScaler y la misma semilla aleatoria (`random_seed = 1`) para que la comparación sea reproducible. La métrica es el rendimiento por resustitución, es decir, exactitud sobre el mismo conjunto con el que se entrenó. Mide capacidad de ajuste, no generalización; eso debe quedar claro porque cualquier modelo se ve mejor de lo que es cuando se evalúa sobre lo que ya vio.

3.2. Implementación 1: Perceptrón clásico

Perceptron.py define la clase base. Los puntos clave:

- `rule(X)` calcula $z = \mathbf{w}^T \mathbf{x} + b$ con `np.dot(X, w[1:]) + w[0]`. El bias se guarda en `w[0]`.
- `predecir(X)` aplica el escalón: `np.where(rule(X_scaled) > 0, 1, 0)`.
- `entrenar(X, y)` inicializa los pesos con $\mathcal{N}(0, 0.01^2)$, recorre epochs épocas y dentro de cada época itera muestra por muestra aplicando la regla delta sólo cuando hay error.

Pseudocódigo del bucle de entrenamiento:

```
for _ in range(epochs):
    for xi, target in zip(X_scaled, y):
        predicted = step(rule(xi))
        if target == predicted:
            continue
        update = eta * (target - predicted)
        w[1:] += update * xi
        w[0] += update
```

Características:

- Actualización online (muestra por muestra).
- Etiquetas en $\{0, 1\}$.
- Sin función de costo continua, así que no hay curva de convergencia suave para diagnosticar.
- Sólo converge si los datos son linealmente separables; si no, oscila.

3.3. Implementación 2: Perceptrón con Descenso por Gradiente

PerceptronGD.py hereda de Perceptron y sobrescribe entrenar con una variante Adaline:

```
y_signed = np.where(y == 1, 1, -1)
for _ in range(epochs):
    net_input = self.rule(X_scaled)
    errores = y_signed - net_input
    w[1:] += eta * X_scaled.T.dot(errores) / n
    w[0] += eta * errores.sum() / n
    costo = (errores ** 2).sum() / 2.0
    costos.append(costo)
```

```
if abs(costos[-2] - costos[-1]) < tol:
    break
```

Diferencias respecto al clásico:

1. **Error continuo.** Usa $y - z$ (salida lineal), no $y - \varphi(z)$. El gradiente queda bien definido y se sigue mejorando los pesos incluso cuando la predicción binaria ya es correcta.
2. **Actualización batch.** El gradiente se promedia sobre todas las muestras por época, en lugar de actualizar muestra por muestra.
3. **Etiquetas en $\{-1, +1\}$.** Simétricas respecto al umbral $z = 0$.
4. **Costo SSE registrado.** Permite graficar convergencia y aplicar *early stopping* cuando $|J_{t-1} - J_t| < \text{tol}$.
5. **η más pequeña.** Como el batch acumula muchas muestras, $\eta = 0.01$ es típico (vs. 0.1 del clásico).

3.4. Implementación 3: Perceptrón con PSO

PerceptronPSO.py también hereda de Perceptron, pero ignora η y sustituye el entrenamiento por una búsqueda de enjambre:

```
particulas = rgen.normal(0, 0.5, (n_particulas, dim))
velocidades = rgen.normal(0, 0.1, (n_particulas, dim))
pbest = particulas.copy()
pbest_fit = [fitness(p, X, y) for p in particulas]
gbest = pbest[argmin(pbest_fit)]

for _ in range(epochs):
    r1, r2 = rand(...), rand(...)
    velocidades = w_inercia*velocidades \
        + c1*r1*(pbest - particulas) \
        + c2*r2*(gbest - particulas)
    particulas += velocidades
    # actualizar pbest/gbest
```

donde $\text{fitness}(w, X, y)$ cuenta las muestras mal clasificadas.

Diferencias clave:

1. **Sin gradientes.** La función de fitness puede ser discontinua (un conteo de errores), cosa que GD no soporta.
2. **Búsqueda poblacional.** 30 partículas exploran el espacio en paralelo. Hay menos riesgo de quedarse atrapado en un mínimo local estrecho.
3. **Hiperparámetros distintos.** Inercia $\omega = 0.7$, $c_1 = c_2 = 1.5$, en lugar de tasa de aprendizaje.
4. **Costo computacional.** Cada iteración evalúa la fitness sobre todas las partículas, $\approx 30 \times$ más cómputo por época que GD.
5. **Sin garantías formales de convergencia**, a diferencia del clásico para datos separables.

3.5. Resultados

Los tres modelos se entrenaron con los mismos datos durante 100 épocas (100 iteraciones para PSO, con $n_{\text{particulas}} = 30$). Resultados de resustitución sobre las 569 muestras:

Modelo	Aciertos	Total	Exactitud
Perceptrón clásico	561	569	98.59%
Perceptrón GD (Adaline)	551	569	96.84%
Perceptrón PSO	558	569	98.07%

Tabla 1: Rendimiento por resustitución de las tres implementaciones.

Detalle de convergencia:

Clásico	Actualización online; sin costo continuo registrado. Converge rápido por la simplicidad de la regla.
GD (Adaline)	Costo SSE inicial: ≈ 292.27 ; Costo SSE final: ≈ 77.71 . 100 épocas usadas (sin disparar early-stopping con $\text{tol}=1e-6$). Decrece monotónicamente: $292 \rightarrow 247 \rightarrow 213 \rightarrow 188 \rightarrow 168 \rightarrow \dots \rightarrow 77.7$
PSO	Errores iniciales del enjambre (gbest): 39 mal clasificados. Tras 100 iteraciones: 11 mal clasificados. Trayectoria: $39 \rightarrow 37 \rightarrow 29 \rightarrow 26 \rightarrow 24 \rightarrow \dots \rightarrow 11$. Estabiliza en 11 errores en las últimas iteraciones.

3.6. Análisis

El perceptrón clásico ganó en aciertos (98.59%), pero conviene leerlo con cuidado: la regla delta sobre datos casi separables produce un hiperplano de margen pequeño muy pegado al training set. Eso ayuda a la métrica por resustitución y suele lastimar la generalización.

GD obtuvo menos aciertos (-1.75 puntos respecto al clásico) porque minimiza un proxy distinto: el SSE sobre la salida lineal. Promedia errores continuos en lugar de empujar los puntos frontera hasta el último voto. Lo que pierde en exactitud lo gana en estabilidad: la curva de costo es interpretable, las garantías de convergencia son formales, y exactamente este algoritmo, con otra activación y muchas capas, es lo que hace funcionar a una red profunda.

PSO se quedó a medio punto del clásico (98.07%) sin usar derivadas en ningún momento. En 100 iteraciones bajó de 39 errores a 11, y luego se estabilizó. El precio es el cómputo: cada iteración requiere evaluar todas las partículas sobre todo el dataset, y los hiperparámetros ω, c_1, c_2 no se ajustan solos. Pero la idea de minimizar un conteo de errores discreto, cosa que el gradiente no puede tocar, es exactamente lo que hace útil a PSO.

Cuándo conviene cada uno:

- **Clásico**: didáctico, o problemas binarios pequeños y casi separables.
- **GD**: cuando se quiere diagnóstico de convergencia, pérdidas diferenciables, o como puente conceptual hacia redes profundas.
- **PSO**: cuando la pérdida no es diferenciable o no convexa, o cuando se quiere explorar globalmente el espacio sin comprometerse a un único punto inicial.

4. Conclusiones

El perceptrón es el mismo modelo en los tres casos: un hiperplano $w^T x + b = 0$ que parte el espacio en dos clases. Lo que cambia es *cómo* se encuentra w , y eso ya cambia bastante.

En este dataset las tres variantes quedan en una franja estrecha, entre 96.84% y 98.59% de aciertos por resustitución. El clásico encabeza (561/569), PSO queda muy cerca (558/569) y GD un poco abajo (551/569). Para clasificación binaria sobre datos casi separables, los tres funcionan; la diferencia real entre ellos no es la métrica.

GD pierde algo de exactitud, pero a cambio entrega una pérdida diferenciable, una curva de costo monótona, y *early stopping* confiable. Por eso es el ancestro directo del entrenamiento por backpropagation. PSO demuestra algo distinto: que se puede entrenar el mismo modelo optimizando una métrica discreta, sin derivadas, a costa de muchas más evaluaciones por época. El clásico, con su regla delta mínima, sigue siendo la referencia teórica.

La conclusión práctica para Sistemas Computacionales es que la arquitectura del modelo y el algoritmo de entrenamiento son independientes. Mismo perceptrón, tres rutinas distintas, tres trayectorias distintas, tres conjuntos distintos de garantías. Saber elegir cuál usar en cada caso es buena parte del trabajo.

Reporte generado a partir de las implementaciones en `Perceptron.py`, `PerceptronGD.py` y `PerceptronPSO.py`.